How to use Piping

The 'Piping' feature in REDCap allows you to inject previously collected data into text on a data collection form or survey, thus providing greater precision and control over question wording. It can also be used in other ways, such as for customizing survey invitations (e.g., by including the respondent's name in the email) or survey completion text (e.g., thanking your respondent by name after completing a survey). To see Piping in action, you may view a live demo of Piping on a survey.

Where can Piping be used?

Piping can be used in many different places in REDCap. Below is a list of all the places where Piping may be utilized to insert a data value *INTO* text.

- Field Label
- Field Note
- Section Header
- Matrix field column headers
- Option labels for multiple choice fields (radio, drop-down, checkbox)
- Slider field labels (i.e. text displayed above slider bar)
- Custom record locking text (if defined, displayed at bottom of form)
- Survey Instructions
- Survey Completion Text
- Survey invitation emails (sent via Participant List or Automated Invitations) - includes both subject and message
- Custom text displayed at top of Survey Queue
- Inside the URL for a survey's 'Redirect to a URL' setting
- Inside the @DEFAULT action tag

From what field types can data be piped?

Data can be piped into the locations listed above *FROM* any type of field in REDCap. If you are piping the answer *FROM* a multiple choice field (e.g., radio, drop-down), by default it will display the option label (not the coded data value) into the location where the field is piped, but for all other field types, it will pipe the literal data value. See below how to append ':value' to the variable name inside brackets to pipe the value (instead of the label) for multiple choice fields. Also see below how to pipe data from checkbox fields, which is slightly different than with other field types.

File Upload fields: File Upload fields have unique piping options. If you are piping *FROM* a File Upload field or Signature field, the field's numerical value will be piped by default, but you can pipe the original filename of the uploaded file by appending the '**:label**' option, such as `[my_field:label]`. Additionally, if you use the '**:inline**' option for a File Upload field, such as `[my_field:inline]`, in which the uploaded file is a PDF file or image file (JPG, JPEG, GIF, PNG, TIF, BMP), the file will be displayed in an inline manner so that it is viewable on the page. (The @INLINE action tag does not need to be used on a field in order to utilize the :inline piping option. So while the @INLINE action tag can be used to display the PDF/image on the field's instrument, the :inline piping option allows you to display the PDF/image in places outside the field's instrument.) Note: Inline images are not able to be displayed inside a downloaded PDF of a survey/instrument that contains data. Additionally, **the :inline option also operates inside emails, so you can pipe a field via :inline inside the email body, thus allowing you to display inline images inside survey invitations or Alerts & Notifications**. If the file being piped via :inline inside an email is not an image file type, it will still attach the file to the email but will display the file's filename in the email text as an alternative to displaying it as an inline image. File Upload fields can also utilize the '**:link**' piping option, such as `[my_field:link]`, where **the file's filename will be displayed as a clickable hyperlink for downloading the file**, which works on webpages and also inside the body of email text (i.e., survey invitations or Alerts & Notifications).

How do I utilize Piping?

Setting up Piping is very simple. Piping does *not* have to be enabled in any way in order to use it. **All you need to do to pipe a data value into any of the valid places listed above is insert into your text the variable name inside square brackets**. For example, if we have a survey question that asks 'What is your first name?', in which the variable name for that question is 'first_name', then the following question may be set up as **'[first_name], what is your date of birth?'**. If the user entered 'John' as their first name, then text for the next question would literally read as **'John, what is your date of birth?'**. In the same way, you could use [first_name] in your survey's survey completion text when the survey is completed, in which **'Thanks for taking our survey, [first_name]!'** would be displayed to the respondent as **'Thanks for taking our survey, John!'**.

If you are piping the answer *FROM* a multiple choice field (radio, drop-down), it will display the option label (not the coded data value) into the location where the field is piped. For example, if you had a drop-down question that said 'What is your favorite ice cream?' with the choices 'Chocolate', 'Vanilla', and 'Strawberry' (all coded as 0, 1, 2, respectively), then after selecting 'Chocolate', the next question would read 'On a scale of 1 to 10, how much to you like Chocolate?'. But for non-multiple choice fields, such as Text, Notes, Slider fields, it will pipe the literal data value. **If you wish to pipe the value (not the label) of a multiple choice field, append ':value'** to the variable name inside brackets - e.g., [race:value]. Note: The usage of ':value' must be used if piping inside the **@DEFAULT Action Tag if you would like to pipe the value of a multiple choice field** as the default value of a field.

If you have a longitudinal project, you may utilize cross-event Piping if you wish. In the same way that cross-event calculations and branching logic are set up, you simply prepend the variable name inside square brackets with the unique event name inside square brackets. For example, if you collected a person's first name on the event named 'Enrollment', which has the unique event name 'enrollment_arm_1', then you could set up piping as **'[enrollment_arm_1][first_name], what is your date of birth?'**. Thus it will pipe the first name value only from the 'Enrollment' event and not any other event, whereas **if the unique event name is not prepended to a variable in a longitudinal project, it will pipe the data value from the current event**.

AM/PM time formats: When piping a time or datetime field, you may convert their time component from military time to 12-hour AM/PM time format by appending the '**:ampm**' option, such as `[visit_time:ampm]`, in which it will display the 12-hour time appended with lowercase 'am' or 'pm'. Note: The seconds component of datetime fields (if applicable) will be removed. Also, any leading zeros will be removed. For example, the value '23:59' would be converted to '11:59pm', and '07:13' will display as '7:13am'.

Piping from Checkbox fields

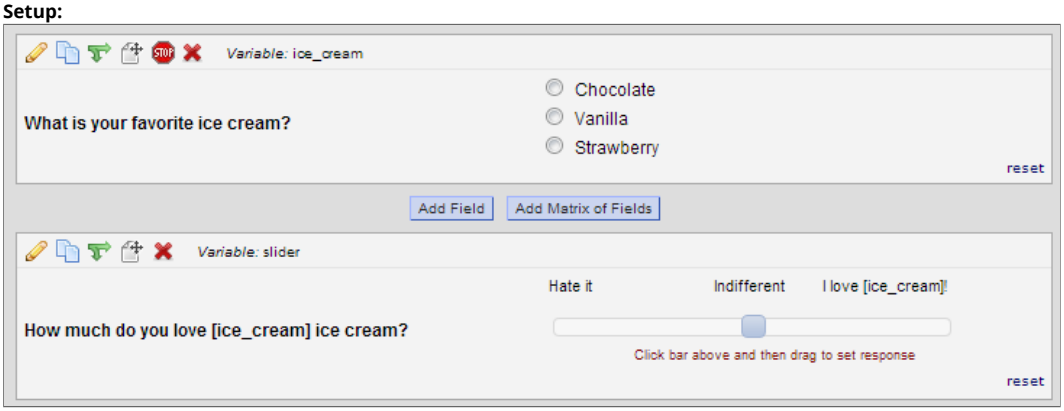
Piping from Checkbox fields is slightly different than with other field types because checkboxes allow for multiple saved values. There are options to display a list of checked choices, unchecked choices, or a specific choice.

- **[my_checkbox:checked]** - Appending ':checked' will display a comma-delimited list of choice labels that have been checked - e.g., 'Sunday, Tuesday, Thursday'. Note: If neither ':checked' nor ':unchecked' is appended to the variable, then it will default to ':checked'.
- **[my_checkbox:unchecked]** - Appending ':unchecked' will display a comma-delimited list of choice labels that have NOT been checked - e.g., 'Monday, Wednesday, Friday, Saturday'.
- **[my_checkbox(code)]** - If a coded value of the checkbox is included inside parentheses after the variable name - e.g., [my_checkbox:(2)] - then it will output the word 'Checked' or 'Unchecked' regarding whether or not that specific choice has been checked off.

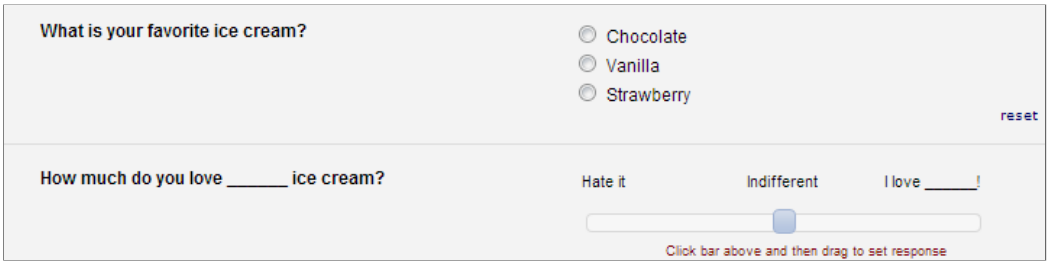
Please note that while the checkbox piping options listed above will return the text labels, you may also **append ':value' to the variable to return the raw value** instead of the label. For example, [my_checkbox:checked:value] and [my_checkbox:unchecked:value] might return '1, 3, 5' and '2, 4, 6, 7', respectively, and [my_checkbox(2):value] will return 1 or 0 if checked or not checked, respectively.

EXAMPLE 1

Setup:



Before:

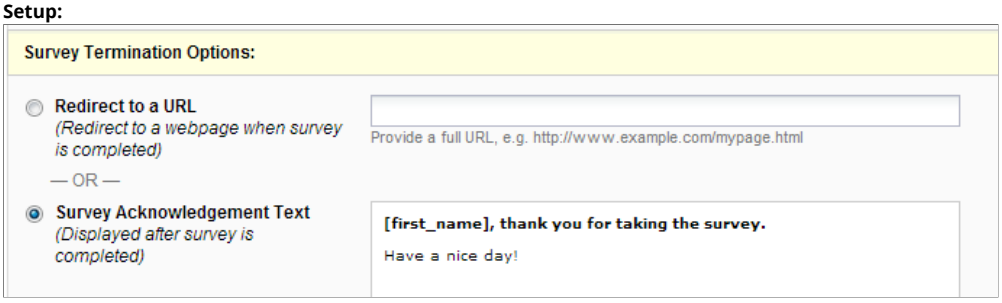


After:

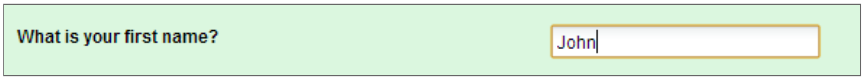


EXAMPLE 2

Setup:



Before:



After:

